SL2DT-06: Dashboard Conversion

> **Series:** SL2DT — Sumo Logic to Dynatrace | **Notebook:** 6 of 11 | **Created:** April 2026 | **Last Updated:** 10/14/2026

Overview

Goal of this step: rebuild in-scope Sumo dashboards in Dynatrace using Notebooks (for analytical dashboards) or Dashboards (for operational/executive views). Map each Sumo panel to a DQL section + visualization. Do **not** migrate every dashboard — only those that survived cut-scope (SL2DT-02).

The translation work leans on SL2DT-04 for the queries and on train-the-trainer for app-team delivery.

Table of Contents

1. What You'll Produce
2. Notebooks vs Dashboards — Choosing the Target
3. Panel-to-Section Mapping
4. Visualization Type Translation
5. Sumo Variables → Dashboard Variables
6. Rebuilding the Core Team's 5 Demo Dashboards
7. App-Team Playbook (train-the-trainer)
8. Step Exit Criteria
9. References

Prerequisites

Requirement	Details
Audience	Core migration engineers + app-team dashboard authors
Inputs	`inventory/dashboards/` + translations from SL2DT-04 + cut-scope from SL2DT-02
Dynatrace access	Platform Token with `document:documents:write` (for notebooks/dashboards)
Prior reading	DASH-01 (Dashboard strategy), dynatrace-notebook-authoring skill

1. What You'll Produce

Artifact	Purpose
`dashboards/notebook-configs/`	Dynatrace Notebook JSON per migrated asset
`dashboards/dashboard-configs/`	Dynatrace Dashboard JSON (for op views)
`dashboards/variable-map.md`	Sumo dashboard variable → Dynatrace dashboard variable mapping
`dashboards/rebuild-tracker.csv`	Per-dashboard status: owner, target, done/in-progress/blocked
`dashboard-rebuild-report.md`	Weekly progress to PM

2. Notebooks vs Dashboards — Choosing the Target

Dynatrace has two dashboard-like products. Pick deliberately.

Feature	Notebook	Dashboard
Purpose	Analytical, runbook, authoring	Operational, at-a-glance view
Navigation	Top-to-bottom reading	Panels in grid
DQL editing	Inline	Inline
Markdown sections	Yes, freely interleaved	Text tiles only
Variables	No — a `$name` reference fails to parse in a notebook section	Yes
Sharing	Document API	Document API
Suited for	Investigation, tutorials, reports	Monitoring displays, SRE pages

Decision Rule

Sumo Dashboard Type	Dynatrace Target
Any dashboard that relies on variables	**Dashboard**
Investigation / analytical	**Notebook**
On-call operator view	**Dashboard**
Team home page	**Dashboard** (with embedded notebook links)
Executive KPI view	**Dashboard**
Post-incident forensics	**Notebook**
SRE runbook with manual steps	**Notebook**

3. Panel-to-Section Mapping

Each Sumo panel becomes one DQL section (with `visualization` field) in the notebook/dashboard.

Sumo Panel Anatomy

```
{
  "id": "panel-123",
  "title": "Error Rate by Service",
  "query": "_sourceCategory=prod/api | timeslice 1m | count by service",
  "visualization": "line",
  "position": { "x": 0, "y": 0, "width": 6, "height": 4 }
}
```

Dynatrace Notebook Section (Version 7)

```
{
  "id": "section-1",
  "type": "dql",
  "title": "Error Rate by Service",
  "state": {
    "input": {
      "value": "fetch logs, from:-1h | filter sumo.source_category ==
\"prod/api\" | makeTimeseries c = count(), interval:1m, by:
{k8s.workload.name}",
      "timeframe": { "from": "now()-1h", "to": "now()" }
    },
    "visualizationSettings": {
      "chartSettings": { "gapPolicy": "connect" }
    }
  },
  "visualization": "lineChart"
}
```

Pick the split field by coverage, not by name. Sumo's `by service` was usually a field your collectors or FERs set, and Dynatrace has no single field that carries it for every log. `service.name` is an OpenTelemetry resource attribute and is empty on most OneAgent-collected logs: on the validation tenant (1 h, 10/14/2026) it was set on 1.3% of logs, `dt.smartscape.service` on 3.6%, `k8s.workload.name` on 41%, and `log.source` on 80%. The example splits by `k8s.workload.name`; for non-Kubernetes sources use `log.source` or `sumo.source_category`. Count `countIf(isNotNull(<field>))` against `count()` on your own logs first — every log without the field lands in one null row.

Mapping Workflow

For each panel:

1. Look up the Sumo query in `translations/all-queries.csv` → get the DQL
2. Add `from:` to the DQL (time range from Sumo panel config)
3. Map Sumo visualization type → Dynatrace visualization (see table below)
4. Preserve title, layout position, description
5. Skip panels marked `retire` in cut-scope

Common Mistakes

- **Copying the time range wrong.** Sumo dashboards have a global time picker that overrides panel-level ranges. Check both.
- **Missing `from:` clause.** The query still runs — it silently uses the tile or notebook timeframe — so it is the #1 cause of numbers that don't match the Sumo panel.
- **Stale DQL from SL2DT-04.** If you re-translated after schema changes, re-pull from the CSV.

4. Visualization Type Translation

Sumo Viz	Dynatrace Viz	Notes
Line chart	Line chart	Most common — direct
Area chart	Area chart	
Bar chart	Bar chart (categorical)	
Column chart	Bar chart (categorical, vertical)	
Pie chart	Pie / donut	Tables read better for many categories
Single value	Single value	With threshold colors
Table	Table	
Honeycomb	Honeycomb	Status grids
Heat map	Heatmap	
Map (geo)	Choropleth map, dot map or bubble map	Pick by data shape: regions, points, or sized points
Gauge	Gauge (or meter bar)	

Sumo Viz	Dynatrace Viz	Notes
Text panel	Markdown section / text tile	

> **Sources:** Edit visualizations (DT docs).

Example — Single Value with Threshold

Sumo: percentage panel showing `error_pct` with red if >5%, yellow if >1%, green otherwise.

Dynatrace Notebook:

```
// 4. Visualization Type Translation — single value with thresholds
fetch logs, from:-5m
| filter sumo.source_category == "prod/api"
| summarize {total = count(), errors = countIf(contains(content, "error",
caseSensitive:false))}
| fieldsAdd error_pct = 100.0 * toDouble(errors) / toDouble(total)
| fields error_pct
```

```
{
  "visualization": "singleValue",
  "visualizationSettings": {
    "thresholds": [
      { "value": 1, "color": "#FFB800" },
      { "value": 5, "color": "#DE350B" }
    ]
  }
}
```

5. Sumo Variables → Dashboard Variables

Sumo dashboards use variables (e.g., `{{environment}}`, `{{host}}`). In Dynatrace, **only Dashboards have variables** — declared in the dashboard and referenced as `$var_name` in tile queries. Notebooks have none: a `$name` reference in a notebook section fails to parse (*"`$` isn't allowed here"*). A Sumo dashboard that depends on variables therefore becomes a Dynatrace Dashboard.

Mapping Example

Sumo variable:

```
{{environment}} → dropdown with values [prod, preprod, dev]
```

Dynatrace dashboard variable: a variable named `environment` whose values are `prod`, `preprod` and `dev` — either a fixed list or a DQL query that returns them — with `prod` as the default.

Usage in DQL:

```
// 5. Sumo Variables → Dashboard Variables
//
// Corrected 08/12/2026: `$variable` references resolve only inside a
Dashboard tile — executed as a
// notebook cell they fail at parse time with "`$` isn't allowed here". The
literals below make the
// cell runnable; the commented line above each filter is the form to paste
into a dashboard tile.
//
// Split by a field your logs carry (see §3). `service.name` was set on 1.3%
of logs on the validation
// tenant (10/14/2026); `k8s.workload.name` on 41%, so ~59% still land in the
null row there. Check the
// null row's share before publishing the tile, and use log.source or
sumo.source_category if it dominates.
fetch logs, from:-1h
// dashboard tile: | filter startsWith(sumo.source_category,
concat($environment, "/"))
| filter isNotNull(sumo.source_category)
| summarize c = count(), by:{k8s.workload.name}
| sort c desc
| limit 20
```

Dependent Variables

Sumo supports variable dependencies (`{{host}}` filtered by `{{environment}}`). In a Dynatrace dashboard, build the dependent variable from a DQL query that references the parent variable.

Multi-Value Variables

Sumo `{{host}}` with multi-select becomes DQL `in(f, $host)` where `$host` is a multi-select dashboard variable.

6. Rebuilding the Core Team's 5 Demo Dashboards

The train-the-trainer model (SL2DT-01 §4) requires the core team to rebuild 5 real-world dashboards as worked examples. Choose:

1. **A simple operational dashboard** — 4–6 panels, all line charts, one service
2. **A multi-team dashboard** — requires variable for team selection
3. **A complex analytical dashboard** — 10+ panels, mixed viz types
4. **An SLA dashboard** — singleValue + thresholds + burn rate
5. **A capacity dashboard** — metric queries, host-level detail

For each, produce:

- The raw Sumo → DT translation (what the translator did)
- The refined DT version (what an engineer polished)
- A README with lessons learned

These become the app-team reference library.

Demo Dashboard Template Structure

```
demos/
├── 01-simple-operational/
│   ├── sumo-original.json       # Exported from Sumo
│   ├── dt-raw-translation.json  # Post-translation, pre-polish
│   ├── dt-final.json            # What shipped
│   └── README.md                # Walkthrough
├── 02-multi-team/
├── 03-complex-analytical/
├── 04-sla/
└── 05-capacity/
```

7. App-Team Playbook (Train-the-Trainer)

Give each app team this playbook:

Dashboard Conversion — 7 Steps

1. **Identify your dashboards** in `cut-scope.md` (filter by your team)
2. **Locate translations** in `translations/all-queries.csv` (filter by your dashboard IDs)
3. **Pick target** — Notebook (analytical) or Dashboard (operational)
4. **Start from a demo template** — pick the closest of the 5 core-team demos
5. **Rebuild panel-by-panel** — for each panel, paste the DQL from translations, set visualization, title, position
6. **Add dashboard variables** if the original had them
7. **Submit PR to core team** for review — include screenshot + link + owner signoff

PR Checklist for App Teams

- [ ] All in-scope panels recreated
- [ ] Each panel's DQL validated (runs successfully against tenant)
- [ ] Variables mapped to dashboard variables
- [ ] Title, description, metadata preserved
- [ ] Screenshot attached showing working state
- [ ] Link to live notebook/dashboard in tenant

Common App-Team Mistakes

- Using `parse` at query time when a FER (SL2DT-03) already extracted the field
- Forgetting `from:` on fetch/timeseries — the query silently uses the tile or notebook timeframe instead, the #1 cause of numbers that don't match the Sumo panel
- Copying Sumo panel time ranges that made sense in Sumo but not in Dynatrace (e.g., Sumo's "all time" → DT needs a concrete `from:`)
- Skipping the cut-scope review ("I want to rebuild everything") — slows them down and burns budget

8. Step Exit Criteria

G6 — Dashboards Rebuilt

- [ ] Core team's 5 demo dashboards complete and committed to `demos/`
- [ ] All Wave 3 (top-usage) dashboards rebuilt and validated for **fidelity** (tiles show the numbers the Sumo original showed)
- [ ] All rebuilt dashboards pass **schema/render validation** — opened in the Dashboards app with zero validation warnings, not just accepted by the API (see **DASH-07 §5**; under SaaS 1.344 a dashboard that fails validation no longer loads at all)
- [ ] App-team Wave 4 dashboards: ≥75% complete, others tracked in `rebuild-tracker.csv`
- [ ] Variable mapping documented for every dashboard that uses variables
- [ ] Every rebuilt dashboard: owner signoff recorded
- [ ] Aggregate metric: dashboard-conversion fidelity ≥90%

Next step: SL2DT-07 — User Governance & Access (IAM groups, bucket-scoped policies, SSO).

---

9. References

Dynatrace visualization surfaces

- Dashboards and notebooks (DT docs)
- Notebooks (DT docs)
- DQL Reference (DT docs)

Sumo Logic dashboard reference (source)

- Sumo Logic dashboards (Sumo Logic docs)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace or Sumo Logic. Always verify information against the official

Dynatrace documentation and Sumo Logic documentation.